

What Is React Three Fiber?

React Three Fiber (R3F) is a **React renderer for Three.js**, allowing you to create powerful 3D experiences using React's component-based syntax. Rather than controlling Three.js scenes manually, you build 3D worlds using JSX and React's state management, hooks, and component paradigms. This makes integration with the wider React ecosystem seamless and development more declarative and organized, especially for those who already know React [\[1\]](#) [\[2\]](#) [\[3\]](#).

Why Use React Three Fiber?

- **Declarative Syntax:** Write 3D graphics and animations the way you write your React components, using JSX.
- **Component Reusability:** Break 3D scenes into reusable, composable React components.
- **React Ecosystem Compatibility:** Use tools like Redux, Zustand, React DevTools, etc.
- **Easier Learning Curve:** Especially for developers already familiar with React, R3F is more approachable than pure Three.js [\[2\]](#).

Key Concepts

1. Canvas

The `<Canvas>` component is the root of your R3F scene. It creates the WebGL renderer, manages the Three.js scene, sets up the render loop, and attaches the canvas to your React tree.

```
import { Canvas } from '@react-three/fiber';

function App() {
  return (
    <Canvas>
      {/* your 3D components here */}
    </Canvas>
  );
}
```

2. JSX Elements = Three.js Objects

Every Three.js class maps to a JSX element (in camelCase), e.g.:

- `<mesh />` maps to `THREE.Mesh`
- `<boxGeometry />` to `THREE.BoxGeometry`
- `<meshStandardMaterial />` to `THREE.MeshStandardMaterial`

Example: Simple Box

```
<Canvas>
  <mesh>
    <boxGeometry args={[2, 2, 2]} />
    <meshStandardMaterial color="orange" />
  </mesh>
</Canvas>
```

- The `args` prop takes an array corresponding to the constructor parameters of the Three.js class.

3. The `attach` Prop

Sometimes, you need to specify where a child attaches on its parent, especially for objects that have multiple attachable properties (e.g., a mesh's geometry and material). The `attach` prop allows explicit connections, though most common cases are inferred automatically^[1]:

```
<mesh>
  <meshStandardMaterial attach="material" color="red" />
  <boxGeometry attach="geometry" args={[1, 1, 1]} />
</mesh>
```

4. Props and State

Any valid property of a Three.js object can be supplied as a React prop.

- Primitive values assign directly (like `color="hotpink"`).
- Object properties (like `position`, `scale`) can be arrays: `position=[^1_1][^1_4][^1_5]`.
- When provided with an array, Fiber will convert/construct a Three.js object as needed^{[1] [4]}.

5. Interactivity

Pointer events work out-of-the-box via raycasting. For example:

```
<mesh onPointerOver={handleHover} onClick={handleClick}>
  <boxGeometry />
  <meshStandardMaterial />
</mesh>
```

6. Using Hooks

You can leverage React hooks for animation and side-effects:

- `useFrame` for frame-by-frame updates, e.g., rotation.
- `useRef` to access and mutate underlying Three.js objects.

Example: Spinning Cube

```
import { Canvas, useFrame } from '@react-three/fiber';
import { useRef } from 'react';

function SpinningBox() {
  const ref = useRef();
  useFrame(() => (ref.current.rotation.x += 0.01));
  return (
    <mesh ref={ref}>
      <boxGeometry args={[1, 1, 1]} />
      <meshStandardMaterial color="orange" />
    </mesh>
  );
}
```

7. Extending Functionality

React Three Fiber works naturally with the `@react-three/drei` library, which provides a large collection of ready-made helpers (controls, environment maps, etc.), and can be combined with code splitting, lazy loading, and use of custom shaders.

8. Limitations and Considerations

- **Requires Dual Expertise:** You should understand both React and basic Three.js concepts (scenes, cameras, meshes) ^[1] ^[2].
- **May abstract away fine control:** For advanced cases, R3F's abstraction sometimes requires a deeper dive or direct Three.js usage ^[2].
- **React-only:** R3F is tied to the React ecosystem ^[2].

Learning Path

1. Prerequisites

- Solid understanding of React (especially hooks).
- Basic knowledge of 3D concepts (meshes, cameras, lights) is beneficial ^[5].

2. Learning Roadmap

- **Set up a project:** Use Vite or Create React App, then install:

```
npm install three @react-three/fiber @react-three/drei
```

- **Start with examples:** Build simple meshes and gradually add interactivity.
- **Explore official documentation and tutorials:** The [React Three Fiber official docs] ^[5], [step-by-step beginner guides] ^[6], and [interactive video tutorials] ^[7] are excellent starting points.
- **Experiment:** Create interactive objects, import models, apply textures, and add controls.

Recommended Resources

- **Official Docs:** [React Three Fiber Documentation]^{[1] [4] [5]}
- **Step-by-step beginner guides:** [Wael Yasmina's beginner tutorial]^[6]
- **Video courses:** [SBCode YouTube/Udemy courses]^{[8] [9]}
- **Interactive playgrounds:** Play with examples and tweak code as you learn^{[10] [7]}.
- **Three.js Journey:** Learn foundational 3D principles—includes R3F content^[5].

Final Tips

- Learn the basics of Three.js—it will unlock much more potential with R3F.
- Use the “Drei” add-on for time-saving helpers and features.
- Embrace React principles: componentization, hooks, and state make complex 3D apps manageable and scalable with R3F.

You'll soon be able to create rich, interactive 3D experiences—all from within React^{[2] [6] [5]}.

✱✱

1. <https://r3f.docs.pmnd.rs/tutorials/how-it-works>
2. <https://graffersid.com/react-three-fiber-vs-three-js/>
3. <https://github.com/pmndrs/react-three-fiber>
4. <https://r3f.docs.pmnd.rs/getting-started/your-first-scene>
5. <https://r3f.docs.pmnd.rs/getting-started/introduction>
6. <https://waelyasmina.net/articles/react-three-fiber-for-beginners/>
7. https://www.youtube.com/playlist?list=PLpepLKamtPjiUF6PvVUbIFhx9HaS0qJs_
8. <https://sbcode.net/react-three-fiber/>
9. <https://www.udemy.com/course/react-three-fiber/>
10. <https://r3f.docs.pmnd.rs/getting-started/examples>